

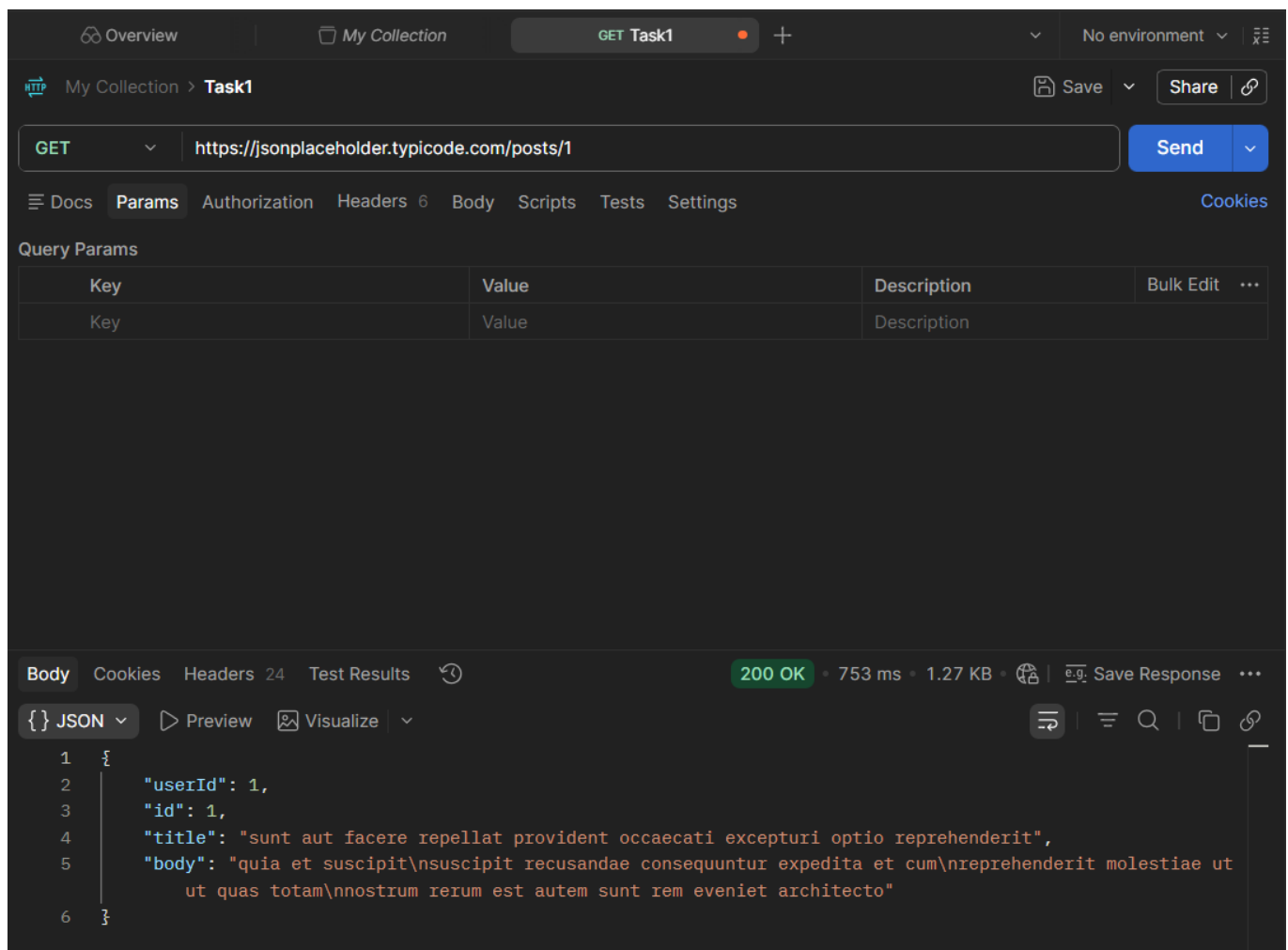
Practice Assignment

API Fundamentals & Postman Mastery

TASKS

TASK 1 Reading Data Without Headers

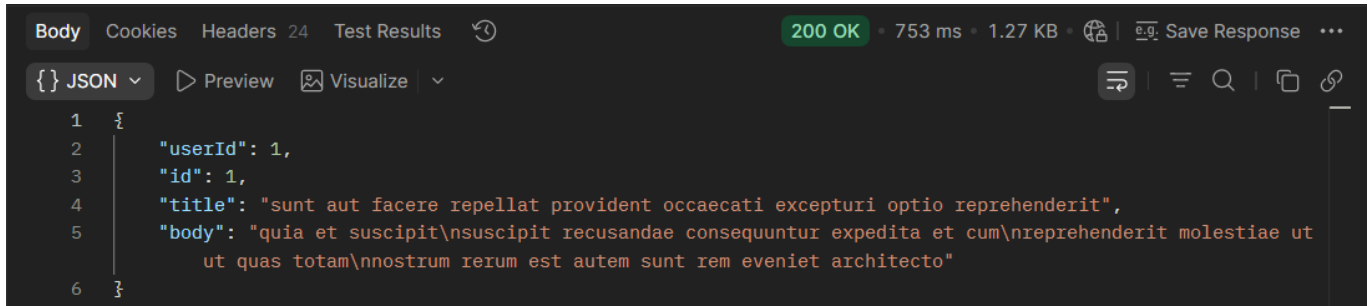
a) Send a GET request to `/posts/1` — leave the Headers and Body tabs completely empty.



The screenshot shows the Postman interface for a GET request to `https://jsonplaceholder.typicode.com/posts/1`. The request is saved in a collection named "Task1". The response is a 200 OK status with a response time of 753 ms and a size of 1.27 KB. The response body is displayed in JSON format, showing a single post object with the following details:

```
1 {
2   "userId": 1,
3   "id": 1,
4   "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
5   "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
6 }
```

b) Record the status code you get back.

A screenshot of a REST client interface. The top bar shows 'Body' selected, with tabs for Cookies, Headers, 24, and Test Results. On the right, it displays '200 OK' in a green box, followed by '753 ms', '1.27 KB', and a 'Save Response' button. Below the top bar, there's a dropdown menu set to 'JSON' and buttons for 'Preview' and 'Visualize'. The main area shows a JSON response with line numbers 1 through 6 on the left. The JSON object has four properties: 'userId' (1), 'id' (1), 'title' (a long Latin string), and 'body' (another long Latin string).

```
1 {  
2   "userId": 1,  
3   "id": 1,  
4   "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",  
5   "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut  
        ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"  
6 }
```

c) In one sentence, explain why this request works even with nothing in the Headers tab.

→ Because GET method is used to retrieve data and it is only method without header so we not need to pass any header it work efficiently without any problem.

TASK 2 Filtering With Query Parameters

a) Send a GET request to /comments, and add query parameter in Params tab: postId = 1.

The screenshot shows the Postman interface with the following details:

- Request:** GET `https://jsonplaceholder.typicode.com/comments?postId=1`
- Params Tab:** A table with one query parameter: `postId` with value `1`.
- Response:** 200 OK, 1.43 s, 1.77 KB. The response body is a JSON array of 3 comments.

Query Params

Key	Value	Description	Bulk Edit	...
✓ <code>postId</code>	<code>1</code>			
Key	Value	Description		

Body (JSON)

```
[
  {
    "postId": 1,
    "id": 1,
    "name": "id labore ex et quam laborum",
    "email": "Eliseo@gardner.biz",
    "body": "laudantium enim quasi est quidem magnam voluptate ipsam eos\ntempora quo necessitatibus\ndolor quam autem quasi\nreiciendis et nam sapiente accusantium"
  },
  {
    "postId": 1,
    "id": 2,
    "name": "quo vero reiciendis velit similique earum",
    "email": "Jayne_Kuhic@sydney.com",
    "body": "est natus enim nihil est dolore omnis voluptatem numquam\nnet omnis occaecati quod ullam at\nvoluptatem error expedita pariatur\nnihil sint nostrum voluptatem reiciendis et"
  },
  {
    "postId": 1,
    "id": 3,
    "name": "odio adipisci rerum aut animi",
    "email": "Nikita@garfield.biz",
    "body": "quia molestiae reprehenderit quasi aspernatur\nnaut expedita occaecati aliquam eveniet"
  }
]
```

b) Confirm the URL automatically updates to include ?postId=1.

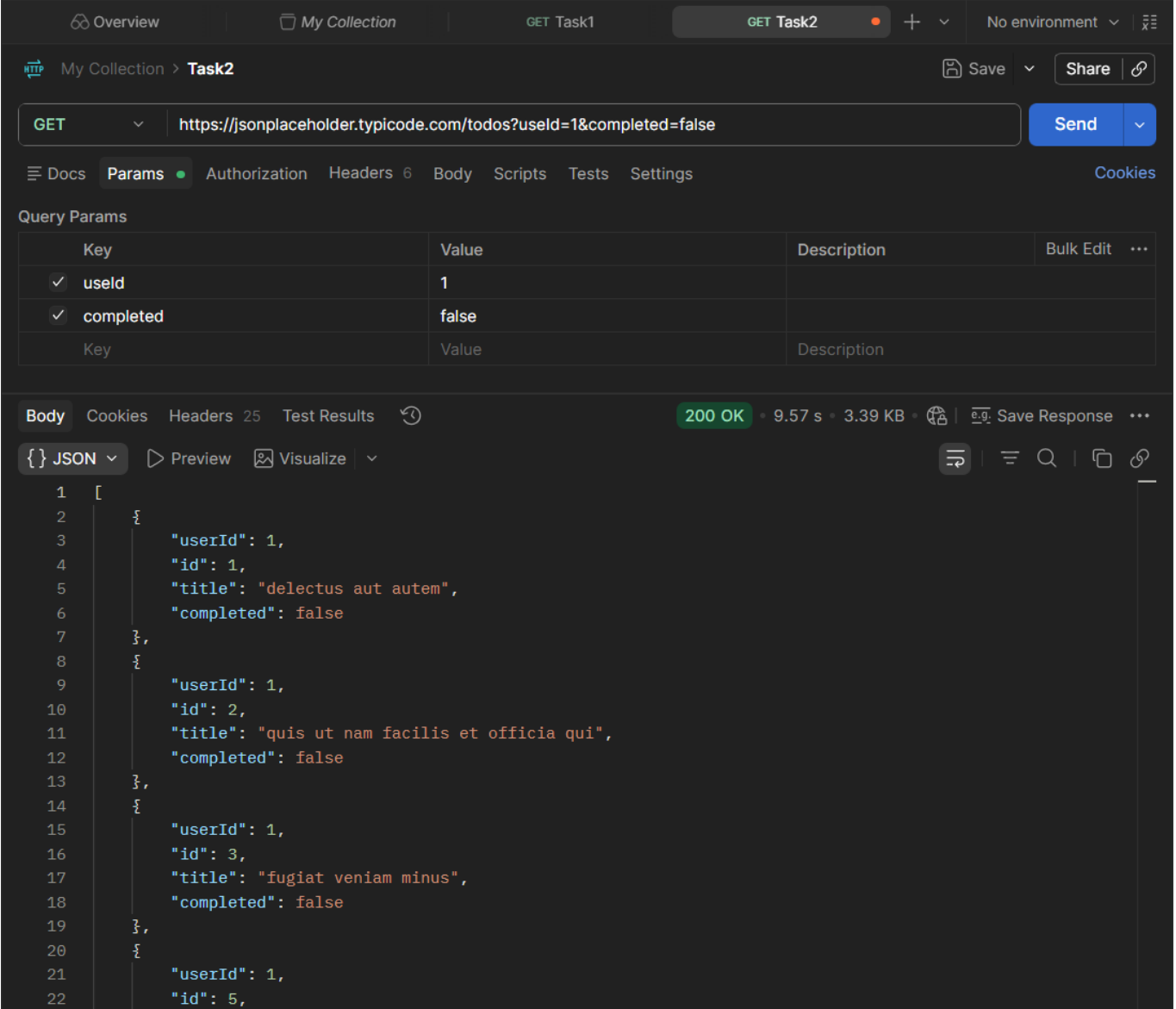
The screenshot shows the Postman interface with the following details:

- Request:** GET `https://jsonplaceholder.typicode.com/comments?postId=1`
- Params Tab:** A table with one query parameter: `postId` with value `1`.

Query Params

Key	Value	Description	Bulk Edit	...
✓ <code>postId</code>	<code>1</code>			
Key	Value	Description		

c) Now combine two filters at once on /todos: `userId = 1` and `completed = false`.



The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** `https://jsonplaceholder.typicode.com/todos?userId=1&completed=false`
- Query Params:**

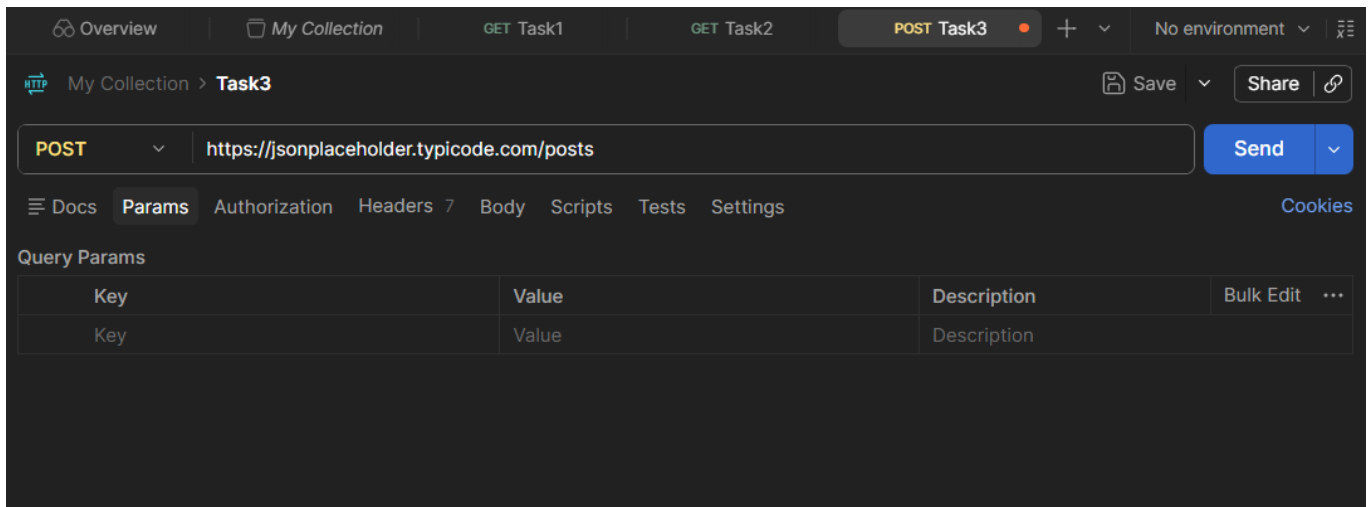
Key	Value	Description
☒ <code>userId</code>	<code>1</code>	
☒ <code>completed</code>	<code>false</code>	
- Response:** 200 OK, 9.57 s, 3.39 KB
- Body (JSON):**

```
[
  {
    "userId": 1,
    "id": 1,
    "title": "delectus aut autem",
    "completed": false
  },
  {
    "userId": 1,
    "id": 2,
    "title": "quis ut nam facilis et officia qui",
    "completed": false
  },
  {
    "userId": 1,
    "id": 3,
    "title": "fugiat veniam minus",
    "completed": false
  },
  {
    "userId": 1,
    "id": 5,
    "title": "et aut vero quos autem",
    "completed": false
  }
]
```

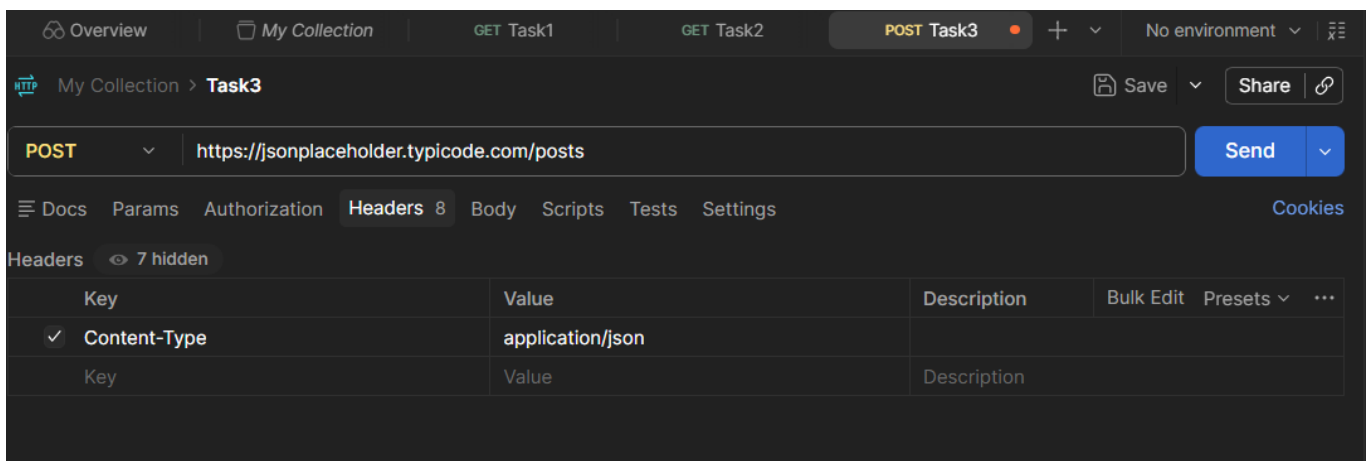
d) In one sentence, explain what changed in the results compared to not using any params.
→ Using query parameters filters the data returned by the server, so only records matching the specified conditions are included instead of all records

TASK 3 Sending Data With Headers & a Body

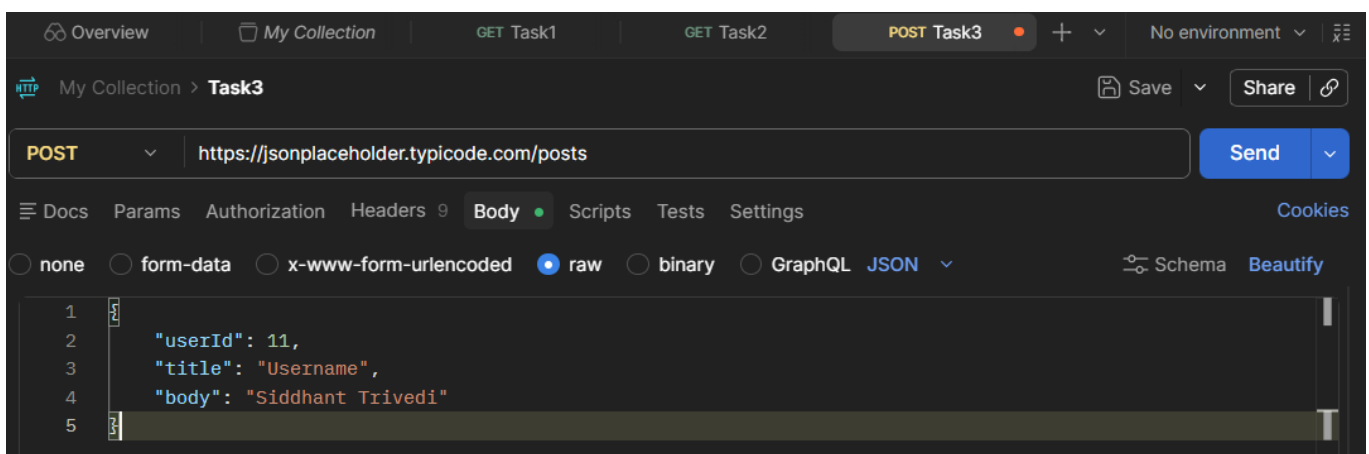
a) Send a POST request to /posts.



b) In the Headers tab, add: Content-Type: application/json



c) In the Body tab (raw, JSON), send a new post with a title, body, and userId of your choice.



d) Record the status code and the new id the server assigned.

The screenshot displays a REST client interface with a dark theme. At the top, a navigation bar includes tabs for 'Overview', 'My Collection', 'GET Task1', 'GET Task2', and 'POST Task3' (which is active). To the right of the tabs are 'No environment' and a menu icon. Below the navigation bar, the breadcrumb 'My Collection > Task3' is shown, along with 'Save' and 'Share' buttons. The main area is divided into two sections. The top section is for the request, showing a 'POST' method and the URL 'https://jsonplaceholder.typicode.com/posts'. Below this are tabs for 'Docs', 'Params', 'Authorization', 'Headers', 'Body' (selected), 'Scripts', 'Tests', and 'Settings'. The 'Body' tab shows a JSON payload:

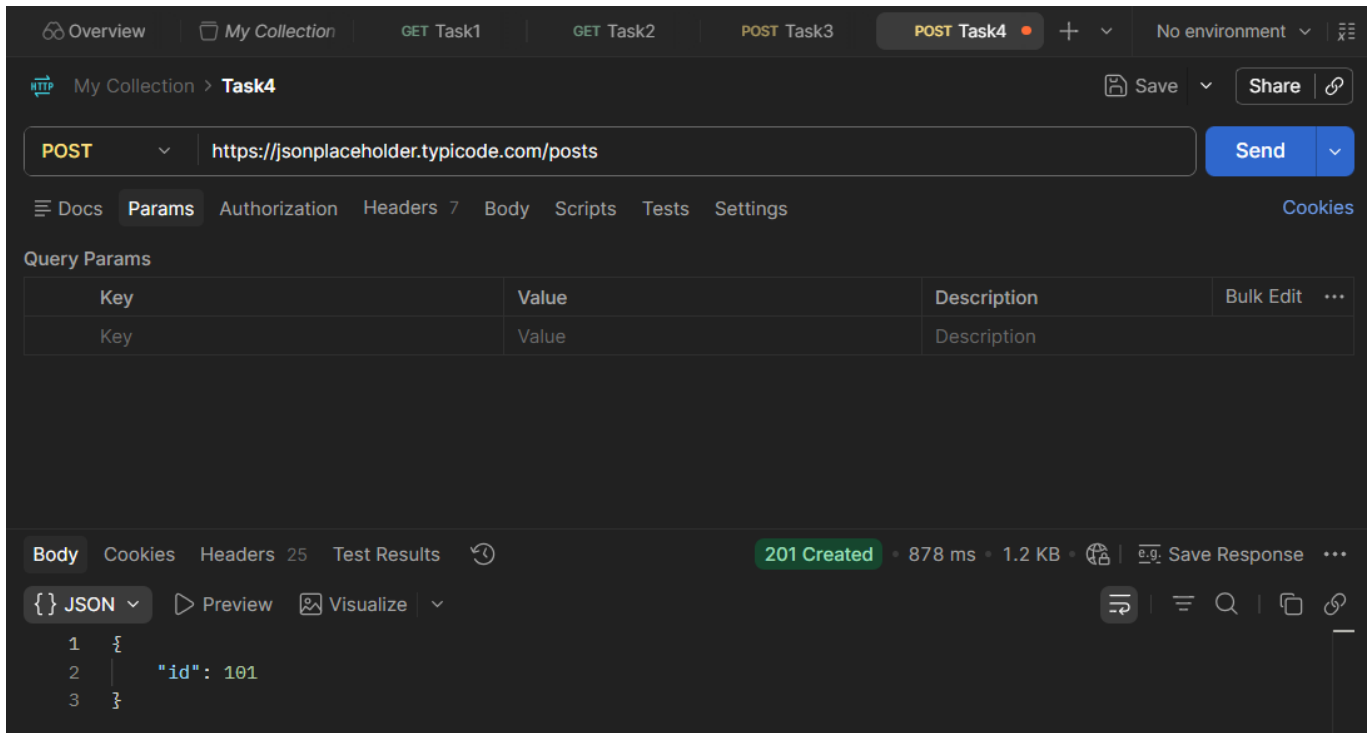
```
{  "userId": 11,  "title": "Username",  "body": "Siddhant Trivedi"}
```

. The bottom section shows the response, with a status bar indicating '201 Created', '715 ms', and '1.28 KB'. Below the status bar are tabs for 'Body', 'Cookies', 'Headers', '25', and 'Test Results'. The 'Body' tab is selected, showing the response JSON:

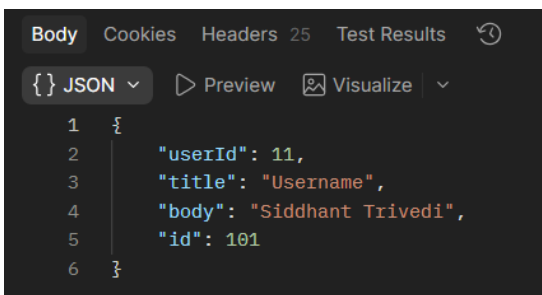
```
{  "userId": 11,  "title": "Username",  "body": "Siddhant Trivedi",  "id": 101}
```

TASK 4 Headers vs. No Headers

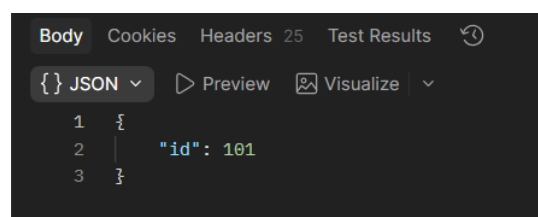
- a) Repeat Task 3's POST request, but this time remove the Content-Type header before sending.



- b) Compare the response to Task 3 — is it different? Record what you observe.



TASK 3 RESPONSE



TASK 4 RESPONSE

→ Both Give Status Code : `201 Created`

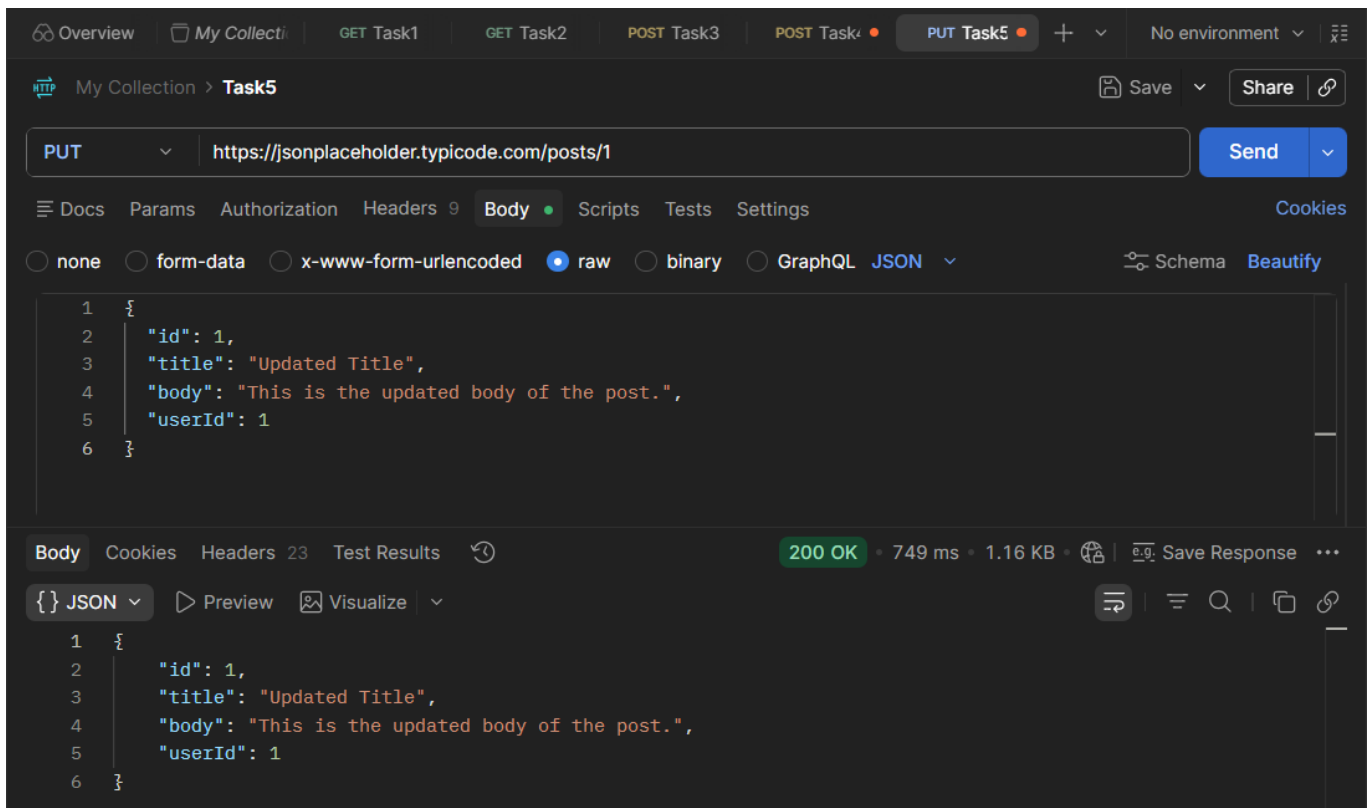
→ The response is the same as Task 3, and the server still returns the created post with id : 101. And this time we not passes Content-Type so it not sawing extra detail which we passed in title, id, body in Task 3.

c) In 2–3 sentences, explain why headers matter for a request like this, even though this practice API is forgiving about it.

→ Headers tell the server how to interpret the data being sent in the request. The Content-Type: application/json header indicates that the request body is in JSON format. JSONPlaceholder accepts the request even without this header because it is a practice API, but real-world APIs often reject or incorrectly process requests if the required headers are missing .

TASK 5 PUT vs. PATCH

- a) Send a PUT request to /posts/1 that replaces the entire post (include id, title, body, and userId in the body).



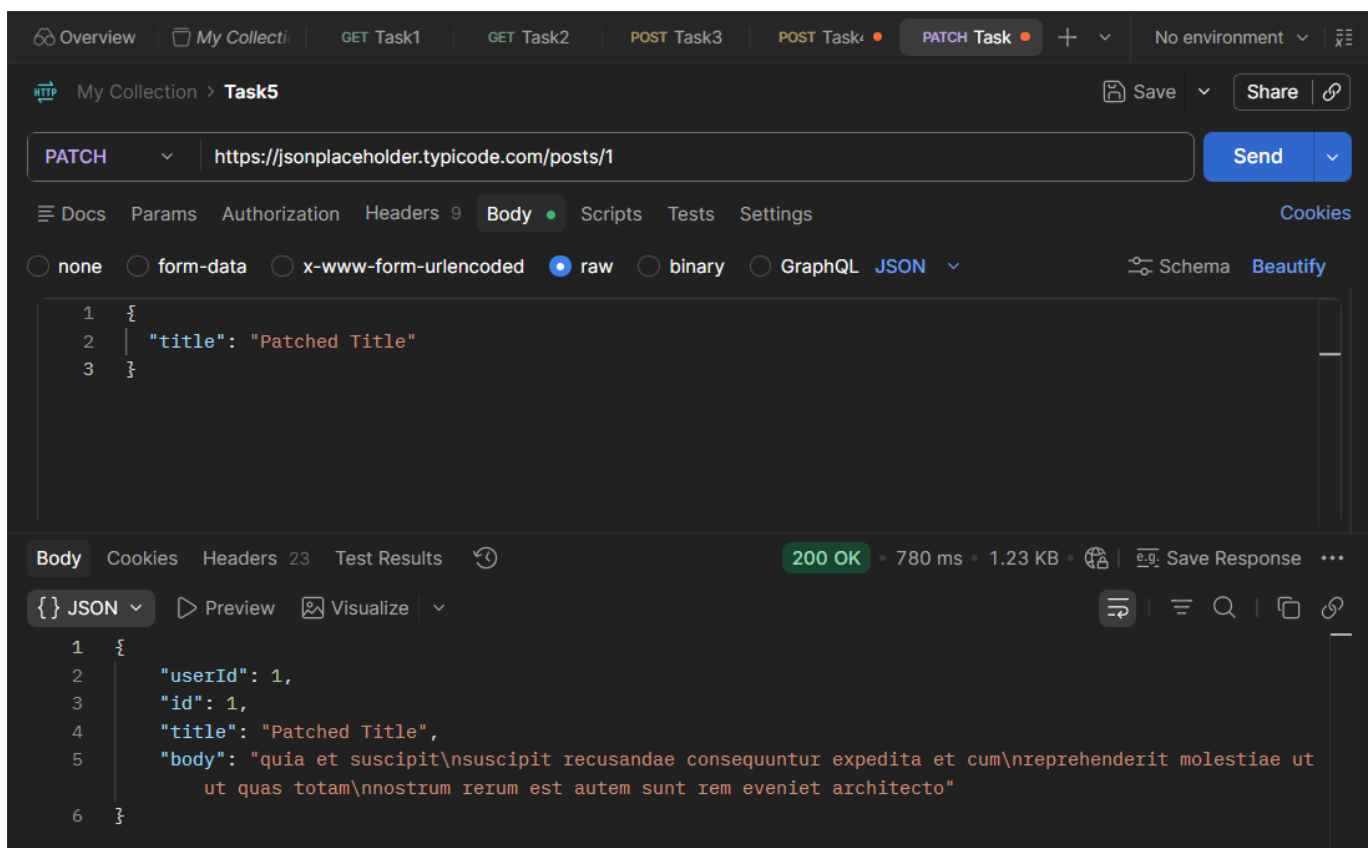
The screenshot shows a REST client interface with a PUT request to `https://jsonplaceholder.typicode.com/posts/1`. The request body is a JSON object with the following structure:

```
1 {
2   "id": 1,
3   "title": "Updated Title",
4   "body": "This is the updated body of the post.",
5   "userId": 1
6 }
```

The response is a 200 OK status with a response time of 749 ms and a body size of 1.16 KB. The response body is a JSON object with the same structure as the request body:

```
1 {
2   "id": 1,
3   "title": "Updated Title",
4   "body": "This is the updated body of the post.",
5   "userId": 1
6 }
```

- b) Send a PATCH request to the same /posts/1 that changes only the title.



The screenshot shows a REST client interface with a PATCH request to `https://jsonplaceholder.typicode.com/posts/1`. The request body is a JSON object with the following structure:

```
1 {
2   "title": "Patched Title"
3 }
```

The response is a 200 OK status with a response time of 780 ms and a body size of 1.23 KB. The response body is a JSON object with the following structure:

```
1 {
2   "userId": 1,
3   "id": 1,
4   "title": "Patched Title",
5   "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
6 }
```

- c) In one sentence, explain the difference between what PUT sent and what PATCH sent.
- PUT sends the complete resource to replace it, whereas PATCH sends only the fields that need to be updated.

Final Challenge — A Realistic Request Flow

TASK 6 The Mini Feedback System

Scenario: you're testing the backend for a simple feedback form, where users read existing comments on a post and can leave their own.

Complete the following steps, in order:

- a) Send a GET request to /comments, using a query parameter to fetch only the comments for postId = 1.

OverviewMy CollectionGET Task1GET Task2POST Task3POST Task4PATCH Task5GET Task6+No environment

My Collection > Task6SaveShare

GEThttps://jsonplaceholder.typicode.com/comments?postId=1Send

DocsParamsAuthorizationHeaders6BodyScriptsTestsSettingsCookies

Key	Value	Description	Bulk Edit	...
✓ postId	1			
Key	Value	Description		

BodyCookiesHeaders26Test Results200 OK • 646 ms • 1.77 KB • Save Response

JSONPreviewVisualize

```
1  [
2    {
3      "postId": 1,
4      "id": 1,
5      "name": "id labore ex et quam laborum",
6      "email": "Eliseo@gardner.biz",
7      "body": "laudantium enim quasi est quidem magnam voluptate ipsam eos\ntempora quo
8      necessitatibus\ndolor quam autem quasi\nreiciendis et nam sapiente accusantium"
9    },
10   {
11     "postId": 1,
12     "id": 2,
13     "name": "quo vero reiciendis velit similique earum",
14     "email": "Jayne_Kuhic@sydney.com",
15     "body": "est natus enim nihil est dolore omnis voluptatem numquam\net omnis occaecati quod ullam
16     at\nvoluptatem error expedita pariatur\nnihil sint nostrum voluptatem reiciendis et"
17   },
18   {
19     "postId": 1,
20     "id": 3,
21     "name": "odio adipisci rerum aut animi",
22     "email": "Nikita@garfield.biz",
23     "body": "quia molestiae reprehenderit quasi aspernatur\naut expedita occaecati aliquam eveniet"
```

- b) Send a POST request to /comments to add a new comment — with no headers at all. Record the status code.

The screenshot shows the Postman interface for a POST request to `https://jsonplaceholder.typicode.com/comments`. The request body is a JSON object:

```
{
  "postId": 1,
  "name": "Siddhant",
  "email": "siddhant@example.com",
  "body": "This is my new comment."
}
```

The response is a 201 Created status with a response time of 888 ms and a size of 1.32 KB. The response body is a JSON object:

```
{
  "postId": 1,
  "name": "Siddhant",
  "email": "siddhant@example.com",
  "body": "This is my new comment.",
  "id": 501
}
```

- c) Resend the same POST from step (b), this time with the Content-Type header included. Compare the two responses.

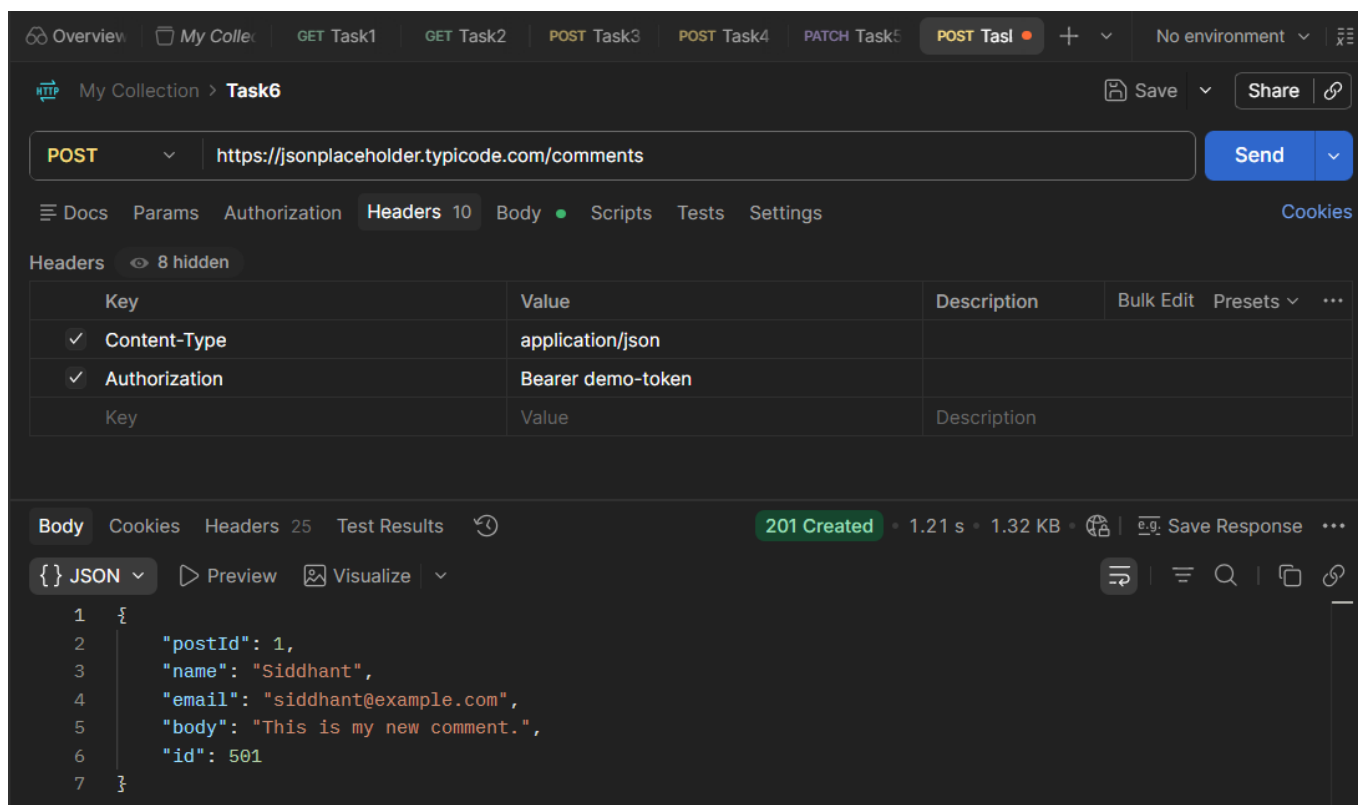
→ Both requests returned the same response and status code because JSONPlaceholder accepts the request even without the Content-Type header.

The screenshot shows the Postman interface for the same POST request, but with the `Content-Type` header set to `application/json`. The response is identical to the previous one, with a 201 Created status and a new id of 501.

Key	Value	Description
✓ Content-Type	application/json	

```
{
  "postId": 1,
  "name": "Siddhant",
  "email": "siddhant@example.com",
  "body": "This is my new comment.",
  "id": 501
}
```

- d) Add one more header to this request: Authorization: Bearer demo-token — to simulate a login-protected submission, even though this practice API won't actually check it.



- e) Record the status code for every step above (a–d) in a short table or list.

Step	Request	Status Code
a)	GET /comments?postId=1	200 OK
b)	POST /comments (No Headers)	201 Created
c)	POST /comments (Content-Type)	201 Created
d)	POST /comments (Content-Type + Authorization)	201 Created

- f) In 2–3 sentences, explain what would happen differently at step (d) if this were a real API that actually required authentication.

→ In a real API, the server would verify the Authorization token before allowing the request. If the token were valid, the request would succeed; otherwise, the server would return an error such as 401 Unauthorized or 403 Forbidden, and the comment would not be created.